

Predicción de popularidad de canciones - Grupo 6

Introducción

El objetivo de este proyecto es desarrollar un modelo predictivo capaz de estimar la popularidad de canciones a partir de características acústicas y metadata disponible. Para ello, se utilizó un conjunto de datos de Spotify que contiene variables numéricas y categóricas asociadas a cada canción. Adicionalmente, se implementó una API para disponibilizar el modelo entrenado, permitiendo realizar predicciones de manera remota.

Análisis preliminar de variables

```
# Aquí vamos a comenzar con las dimensiones de los conjuntos de datos

print("Dimensiones dataTraining:", dataTraining.shape)
print("Dimensiones dataTesting:", dataTesting.shape)

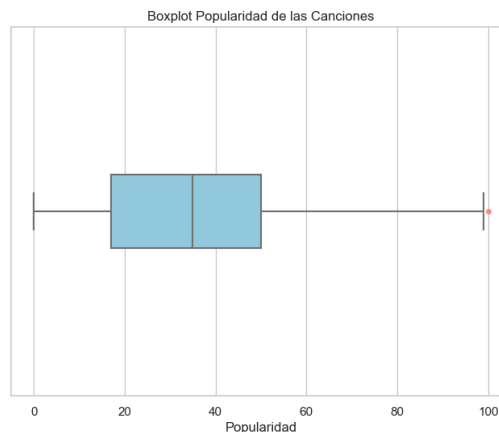
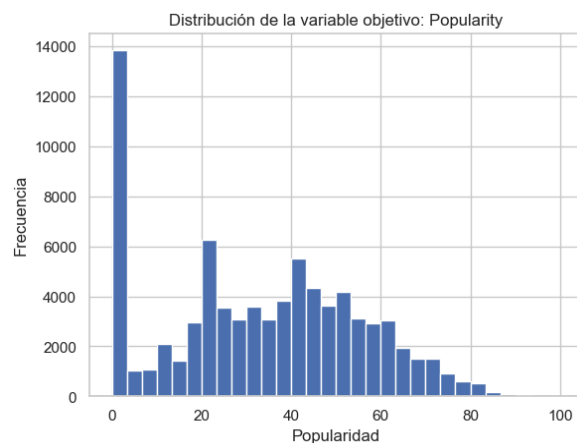
✓ 0.0s

Dimensiones dataTraining: (79800, 21)
Dimensiones dataTesting: (34200, 19)
```

El conjunto de datos de entrenamiento cuenta con 79,800 observaciones y 21 variables, mientras que el conjunto de prueba contiene 34,200 observaciones con 2 variables menos, las cuales son 'Unnamed: 0' (contiene el índice de la tabla) y 'popularity' la cual no se incluye en las competencias de Kaggle.

```
<class 'pandas.DataFrame'>
RangeIndex: 79800 entries, 0 to 79799
Data columns (total 21 columns):
#   Column              Non-Null Count  Dtype  
---  -
0   Unnamed: 0           79800 non-null  int64  
1   track_id             79800 non-null  str    
2   artists              79800 non-null  str    
3   album_name           79800 non-null  str    
4   track_name           79800 non-null  str    
5   duration_ms          79800 non-null  int64  
6   explicit             79800 non-null  bool    
7   danceability          79800 non-null  float64 
8   energy               79800 non-null  float64 
9   key                  79800 non-null  int64  
10  loudness              79800 non-null  float64 
11  mode                  79800 non-null  int64  
12  speechiness           79800 non-null  float64 
13  acousticness          79800 non-null  float64 
14  instrumentalness       79800 non-null  float64 
15  liveness              79800 non-null  float64 
16  valence               79800 non-null  float64 
17  tempo                 79800 non-null  float64 
18  time_signature        79800 non-null  int64  
19  track_genre           79800 non-null  str    
20  popularity            79800 non-null  int64  
dtypes: bool(1), float64(9), int64(6), str(5)
memory usage: 12.3 MB
```

Análisis de la variable objetivo

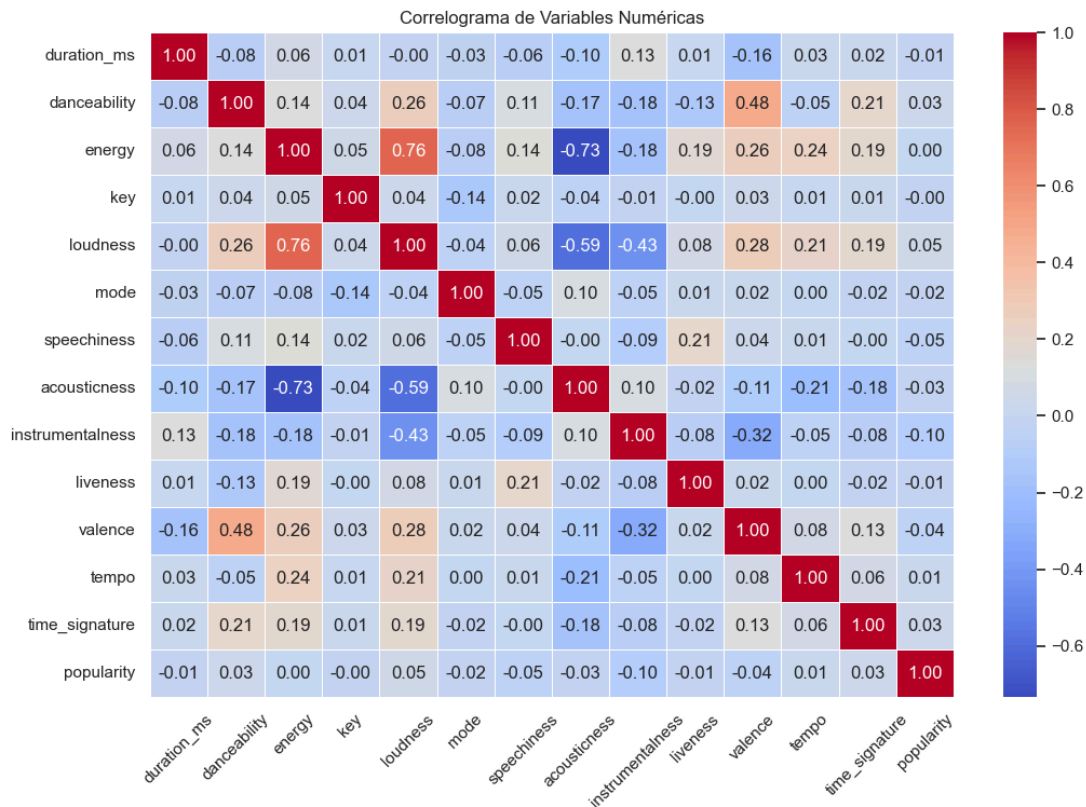


A partir de la inspección inicial, se identificó que no existen valores faltantes en las variables, lo que simplifica el proceso de preprocesamiento. Las variables numéricas

incluyen características relacionadas con la duración, propiedades acústicas de las canciones, energía, clave musical, volumen, entre otras características cuantitativas. También incluye variables categóricas como el género musical, id y nombre de las canciones, artistas y álbumes. Finalmente se observa una variable de tipo bool, la cual puede ser transformada en numérica más adelante.

A partir de los gráficos anteriores, se observa que los 79800 registros se distribuyen mayormente entre el rango de 0 a 80 con un pico cercano a 14000 canciones ubicadas en la popularidad de 0 y 3. El percentil 25 se encuentra en 17, la mediana se encuentra en 35, el percentil 75 en 50 y su valor máximo toma el valor de 100 mostrado como dato atípico en el gráfico de cajas y bigotes. Se observa que es muy raro tener una canción con el mayor puntaje de popularidad, teniendo en cuenta que el 75% de los datos tienen una popularidad menor o igual a 50.

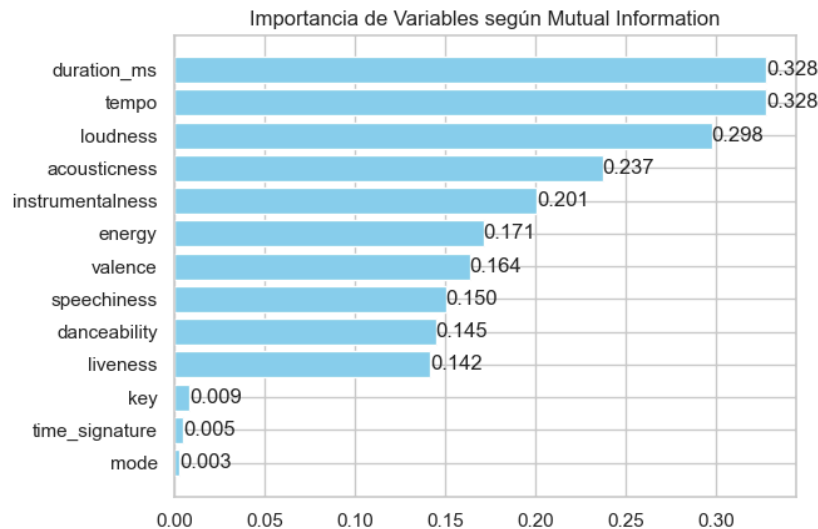
Relación de variables numéricas



Analizando el correlograma anterior, se observa que 'loudness' y 'energy' tienen una relación lineal positiva de 0.76. Adicionalmente las variables 'acousticness' y 'energy' tienen una relación lineal negativa de -0.73. Con lo anterior, se puede probar el modelo de ML prescindiendo la variable 'energy' para no tener variables predictoras redundantes. Finalmente se observa que la variable objetivo no tiene una correlación marcada con las variables predictoras numéricas.

Método de selección de variables: Mutual information

A diferencia del coeficiente de correlación de Pearson, que solo detecta relaciones lineales, el método de 'mutual information' es capaz de medir relaciones lineales y no lineales entre variables. Para este caso, es de utilidad para determinar cuánto nos revela una variable predictora sobre la variable respuesta.



El gráfico anterior informa que las variables 'duration_ms', 'tempo' y 'loudness' tienen la mayor importancia sobre la variable objetivo. Por otro lado, las variables 'key', 'time_signature' y 'mode' no tienen gran importancia sobre la variable respuesta, se puede considerar no incluirlas en el modelo de ML.

Pre-procesamiento de datos

Los datos fueron divididos en conjuntos de entrenamiento y validación con el fin de evaluar la capacidad de generalización del modelo. Se construyó un pipeline que integra el preprocesamiento y el modelo predictivo.

Entrenamiento del Modelo Random Forest

Se entrenó un modelo baseline basado en Random Forest para establecer un punto de referencia. Posteriormente, se realizó la calibración de hiperparámetros, obteniendo un modelo con mejor desempeño.

Para las variables numéricas se utilizó imputación mediante la mediana, mientras que

```
[34] ✓ 27s # Aquí vamos con el entrenamiento del modelo baseline
model_rf.fit(X_train, y_train)
```

```
[28] # Aquí vamos a dividir nuestros datos en los conjuntos de entrenamiento y validación
X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.2, random_state=42)]

[29] # Este es nuestro pipeline de preprocesamiento para variables numéricas y categóricas

numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median'))])

categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore'))])

preprocessor = ColumnTransformer(transformers=[
    ('num', numeric_transformer, numeric_features),
    ('cat', categorical_transformer, categorical_features)])

[32] # Aquí va nuestro pipeline completo con modelo baseline Random Forest

model_rf = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('regressor', RandomForestRegressor(
        random_state=42,
        n_estimators=50,
        max_depth=10,
        n_jobs=-1))])
```

para las variables categóricas se aplicó imputación por la moda seguida de codificación one-hot. Este enfoque garantiza consistencia en las transformaciones y evita fugas de información entre entrenamiento y validación.

Calibración del modelo

La calibración de hiperparámetros se realizó mediante RandomizedSearchCV, lo cual permitió explorar diferentes configuraciones del modelo de forma eficiente.

Dado el tamaño del dataset y el costo computacional, la búsqueda se llevó a cabo

```
[34] # Aquí vamos con el entrenamiento del modelo baseline
model_rf.fit(X_train, y_train)

... Pipeline
  ├── preprocessor: ColumnTransformer
  │   ├── num
  │   │   └── SimpleImputer
  │   └── cat
  │       ├── SimpleImputer
  │       └── OneHotEncoder
  └── RandomForestRegressor

[36] # Aquí vamos predicciones sobre el conjunto de validación
y_pred_val = model_rf.predict(X_val)

[37] # Aquí vamos con la evaluación del modelo baseline
mae = mean_absolute_error(y_val, y_pred_val)
rmse = np.sqrt(mean_squared_error(y_val, y_pred_val))
r2 = r2_score(y_val, y_pred_val)
print("MAE:", mae)
```

```
print("R2 ajustado:", r2_best)

MAE ajustado: 16.338457813949592
RMSE ajustado: 20.263988628281314
R2 ajustado: 0.16946270706639543

El modelo calibrado presentó una mejora respecto al baseline en todas las métricas evaluadas. El MAE disminuyó a 16.34 y el RMSE a 20.26, lo que indica una reducción en el error de predicción. Asimismo, el coeficiente R2 aumentó a 0.17, lo que refleja una mayor capacidad del modelo para explicar la variabilidad de la popularidad. Estos resultados confirman que la calibración de hiperparámetros contribuyó positivamente al desempeño del modelo.

[44] # De acuerdo con nuestro resultado a aquí vamos con el reentrenamiento del mejor modelo con todos los datos de entrenamiento
best_model.fit(X, y)

... Pipeline
  ├── preprocessor: ColumnTransformer
  │   ├── num
  │   │   └── SimpleImputer
  │   └── cat
  │       ├── SimpleImputer
  │       └── OneHotEncoder
  └── RandomForestRegressor
```

sobre una muestra representativa del conjunto de entrenamiento. Se evaluaron parámetros como el número de árboles (`n_estimators`), la profundidad máxima (`max_depth`) y los criterios de división, los cuales influyen directamente en la complejidad y capacidad de generalización del modelo.

Desempeño del modelo Random Forest

El modelo calibrado presentó un error absoluto medio (MAE) de 16.34 y en el error cuadrático medio (RMSE) de 20.26, así como un incremento en el coeficiente de

determinación R^2 de 0.16. aunque este último resultado no cobra mucha importancia frente a los modelos de ensamblaje por ser de naturaleza no lineal. Los resultados del modelo Random Forest con la calibración de hiperparámetros evidencia un RMSE alto, ya que se puede afirmar que puede equivocarse en 20 puntos su predicción. Por lo anterior, se buscará otro modelo que sea capaz de responder mejor a la popularidad de las canciones.

Entrenamiento del modelo XGBoost

Adicionalmente al modelo de Random Forest aplicado decidimos hacer una prueba adicional usando el modelo de XGradientBoosting, buscando encontrar un algoritmo que logre predecir mejor la popularidad de determinada canción; además decidimos hacer un esfuerzo de feature engineering combinando las principales métricas o las más intuitivas con el fin de usar al máximo el poder explicativo de nuestra información.

Concretamente creamos las métricas:

Multiplicativas

- Energy and Danceability
- Acousticness and Energy
- Valence Energy

Limpieza de la columna artista para obtener solamente el artista principal(el primero) de cada canción.

```
] # Creamos nuevas variables buscando capturar mejor el efecto multiplicativo de los diferentes atributos de las canciones
dataTraining['energy_danceability'] = dataTraining['energy'] * dataTraining['danceability']
dataTraining['valence_energy'] = dataTraining['energy'] * dataTraining['valence']
dataTraining['acousticness_energy'] = dataTraining['acousticness'] * dataTraining['energy']
# Dado que tenemos varios artistas de una misma canción extraeremos el primero para entender cuál es el artista principal y buscar comportamientos asociados
dataTraining['main_artist'] = dataTraining['artists'].str.split(pat=';').str[0]
# Convertimos la variable duration en ms a minutos y luego aplicamos Logaritmo
dataTraining['duration_min'] = dataTraining['duration_ms'] / 60000
dataTraining['duration_log'] = np.log1p(dataTraining['duration_min'])
```

Adicionalmente establecimos métricas de la popularidad promedio de cada uno de los géneros y artistas buscando entender mejor el efecto de estas variables categóricas sobre el modelo. Para no caer en Sobreajuste calculamos estas métricas sobre los datos de Train en cada uno de los cortes del CV.

Para la etapa de validación entrenamos dos modelos sin ajuste de hiper parámetros, inicialmente un LightGB y posteriormente un XG Boosting, buscando cuál de los dos nos ofrecía mejor desempeño y el tiempo que nos tomaba entrenar uno y otro. Los resultados son de RMSE de 15.094 y 14.6 para los modelos de LightGB y XG Boosting

respectivamente; ambos resultados muy superiores a los que habíamos logrado con nuestro modelo inicial basado en Random Forest.

El proceso de calibración lo haremos con la función `RandomizedCVSearch` que como recordamos es una búsqueda aleatoria por el universo de parámetros, mucho mejor en términos de rendimiento para la máquina y con resultados muy similares al método exhaustivo.

```
#Corremos también el modelo XGBoost para tener una idea de qué modelo tiene mejor desempeño

scores = cross_val_score(
    pipeline_xgb,
    X[features],
    y,
    cv=5,
    scoring='neg_root_mean_squared_error'
)

print("RMSE por fold:", -scores)
print("RMSE promedio:", -scores.mean())

RMSE por fold: [14.69898891 15.01761627 14.38467598 14.59965992 14.66798401]
RMSE promedio: 14.6737850189209
```

Al comparar los desempeños encontramos que los tiempos de ejecución son muy similares y el modelo XGBoosting ofrece un desempeño ligeramente superior por lo que hacemos el fine tuning de parámetros sobre este modelo obteniendo un RMSE de 13.79, el mejor resultado que hemos tenido hasta el momento por lo que este es el modelo que usaremos para la competencia.

```
#Imprimimos los resultados del fine tuning de parámetros
print("Mejores hiperparámetros:", search.best_params_)
print("Mejor RMSE:", -search.best_score_)

Mejores hiperparámetros: {'modelo__num_leaves': 63, 'modelo__n_estimators': 300, 'modelo__max_depth': 10, 'modelo__learning_rate': 0.1}
Mejor RMSE: 13.792486381530761
```

Disponibilización del modelo en API alojada en la nube

Se creó un archivo llamado `custom_transformers.py` para guardar una clase que transforma las variables categóricas de artista, género y álbum en numéricas usando promedios, desviaciones y conteos de la popularidad sobre dichas variables. Este archivo será llamado con el procedimiento `import` sobre el notebook donde se crea el modelo escogido. Utilizando el modelo XGBoost calibrado, se exporta y guarda el modelo en un archivo `.pkl` en una carpeta llamada `'flask_api'` de forma local junto con el archivo `'custom_transformers'`.

```
modeloxgb = search.best_estimator_  
✓ 0.0s Python  
  
# Exportar modelo a archivo binario .pkl  
joblib.dump(modeloxgb, 'C:/Users/areva/OneDrive - Universidad de los Andes/Documents/GitHub/MIAD_MLP_2026/Semana 4/flask_api/popularidad.pkl', con  
✓ 0.0s Python
```

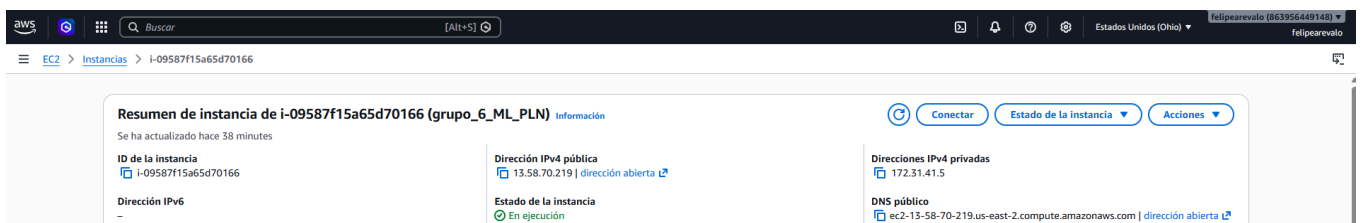
Posteriormente se crea el archivo prediction.py, el cual contiene la función que realiza la predicción utilizando el archivo popularidad.pkl el cual recibe la predicción y la transforma en un dataframe.

```
def predict(data_dict):  
    path = os.path.join(os.path.dirname(__file__), 'popularidad.pkl')  
    modeloxgb = joblib.load(path)  
  
    df = pd.DataFrame([data_dict])  
  
    for col in df.columns:  
        if df[col].dtype == 'object':  
            df[col] = df[col].astype('category')  
  
    p1 = modeloxgb.predict(df)[0]  
    return float(p1)
```

Posteriormente, en el archivo Api.py se definen las entradas que tendrá la herramienta, a partir de la selección de variables utilizadas en este modelo. Se modifica el tipo de cada argumento según corresponda en cada una y finalmente llama a la función .predict para que se aplique sobre el diccionario. Finalmente se prueba en local para replicar posteriormente en el servidor en la nube.

```
PROBLEMS 8 OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER  
127.0.0.1 - - [26/Apr/2026 17:04:55] "GET / HTTP/1.1" 200 -  
127.0.0.1 - - [26/Apr/2026 17:04:55] "GET /swaggerui/droid-sans.css HTTP/1.1" 304 -  
127.0.0.1 - - [26/Apr/2026 17:04:55] "GET /swaggerui/swagger-ui.css HTTP/1.1" 304 -  
127.0.0.1 - - [26/Apr/2026 17:04:55] "GET /swaggerui/swagger-ui-bundle.js HTTP/1.1" 304 -  
127.0.0.1 - - [26/Apr/2026 17:04:55] "GET /swaggerui/swagger-ui-standalone-preset.js HTTP/1.1" 304 -  
127.0.0.1 - - [26/Apr/2026 17:04:56] "GET /swagger.json HTTP/1.1" 200 -  
127.0.0.1 - - [26/Apr/2026 17:05:49] "GET /predict/?album_name=love%20song&danceability=0.7&energy=0.4&loudness=0.2&speechiness=0.1&acousticness=0.5&instrumentalness=0.3&liveness=0.3&valence=0.5&tempo=1&track_genre=rock&energy_danceability=0.1&valence_energy=0.8&acousticness_energy=0.4&main_artist=que  
en&duration_min=3 HTTP/1.1" 200 -
```

Se creó la instancia EC2 en AWS para correr el modelo como servicio en la nube. Al tener la instancia creada, se importaron las librerías comunes como pandas, scikit-learn, el modelo seleccionado xgboost y también las librerías para ejecutar la Api como joblib y flask.



Posteriormente se clonó el repositorio donde están alojados los archivos api.py, custom_transformers.py u prediction.py. Se corrigió un problema de versionamiento de

pandas y se ejecutó la api desde el servidor en la nube. Se utilizaron los 2 primeros registros del conjunto de test en el servidor en la nube.

curl -X 'GET' \

```
'http://13.58.70.219:5000/predict/?album_name=Hillsong%20Worship&danceability=0.369&energy=0.598&loudness=-6984&speechiness=0.0304&acousticness=0.00511&instrumentalness=0.0611&liveness=0.176&valence=0.0466&tempo=120.0' -H 'accept: application/json'
```

Request URL

```
http://13.58.70.219:5000/predict/?album_name=Hillsong%20Worship&danceability=0.369&energy=0.598&loudness=-6984&speechiness=0.0304&acousticness=0.00511&instrumentalness=0.0611&liveness=0.176&valence=0.0466&tempo=120.0
```

Server response

Code	Details
200	Response body <pre>{ "result": 38.115821838378906 }</pre> Response headers <pre>connection: close content-length: 37 content-type: application/json date: Sun, 26 Apr 2025 23:26:34 GMT server: Werkzeug/3.0.1 Python/3.12.3</pre>

Response body

```
{  
  "result": 38.115821838378906  
}
```

curl -X 'GET' \

```
'http://13.58.70.219:5000/predict/?album_name=Grieving%20Birth&danceability=0.171&energy=0.997&loudness=-3586&speechiness=0.118&acousticness=0.00521&instrumentalness=0.001&liveness=0.42&valence=0.0294&tempo=120.0' -H 'accept: application/json'
```

Request URL

```
http://13.58.70.219:5000/predict/?album_name=Grieving%20Birth&danceability=0.171&energy=0.997&loudness=-3586&speechiness=0.118&acousticness=0.00521&instrumentalness=0.001&liveness=0.42&valence=0.0294&tempo=120.0
```

Server response

Code	Details
200	Response body <pre>{ "result": 22.00096321105957 }</pre> Response headers <pre>connection: close content-length: 36 content-type: application/json date: Sun, 26 Apr 2025 23:30:18 GMT server: Werkzeug/3.0.1 Python/3.12.3</pre>

Response body

```
{  
  "result": 22.00096321105957  
}
```

Se observa que ambas canciones no tienen gran puntaje de popularidad, haciendo énfasis que la segunda canción es mucho menos popular teniendo alto valor en el volumen/ruido y estar clasificado en un género poco conocido a nivel general.

Conclusiones

Del desarrollo de nuestro proyecto concluimos que:

- El EDA es uno de los pasos fundamentales para cualquier proyecto de analítica, nos ayuda a entender el contenido de los datos, sus distribuciones, las relaciones entre estos y el aporte potencial de cada variable al modelo que estamos construyendo. Es fundamental hacer este ejercicio de forma rigurosa para obtener mejores resultados, identificar qué información es potencialmente valiosa y cuál puede ser ruido que no aporta nada a nuestra etapa de modelado.
- El Fine tuning de hiperparámetros es un procedimiento fundamental para exprimir al máximo el rendimiento de nuestros modelos, tanto en RF como en XGBoosting fue evidente la mejora de los resultados tras hacer el fine tuning.
- El Feature Engineering también se presenta como una técnica fundamental para lograr los mejores resultados, hacia adelante sería muy interesante profundizar sobre este tema en la clase, pues nos daría muchas herramientas para sacar el máximo provecho posible a la data.
- Desplegar el modelo de ML como un servicio de Api en la nube facilita el uso de las soluciones desde cualquier dispositivo conectado a internet. En la práctica se destaca el beneficio de integrarlo a otras Apis y entornos que hacen parte de la organización.
- Se consideró mantener las variables categóricas como 'Artista', 'Género' y 'Album', fijando criterios técnicos para no caer en la 'maldición de la dimensionalidad'. Cobra sentido cuando existen géneros musicales más populares que otros, y a su vez, artistas y álbumes que tienen un gran renombre detrás más que los atributos específicos de sus canciones.
- En aplicaciones reales, es fundamental entender que un muy buen modelo de Machine learning no es suficiente si no se sabe disponibilizar a través de una API para el acceso y uso del negocio. Este tipo de aplicaciones de negocio requieren escalabilidad y disponibilidad, elementos que representan una ventaja en contextos de alta competitividad